

Relatório de Benchmark — SGBDs Relacionais

Resumo interno para entender o que foi testado, o que esperávamos e o que os dados mostraram. Não é o relatório final.

1. O que estamos analisando

Três bancos de dados relacionais com **arquiteturas internas completamente diferentes**. A pergunta central é: *essa diferença de arquitetura afeta o desempenho? Em quais situações?*

Banco	Paradigma	Como funciona
PostgreSQL	Um processo por conexão	Cada cliente vira um processo separado no SO. Isolamento total, mas custo maior de memória.
MySQL (InnoDB)	Threads dentro de um processo	Um processo central atende todos os clientes via threads compartilhadas. Mais leve por conexão.
SQLite	Embarcado / sem servidor	Não existe "servidor". O código do banco roda dentro da própria aplicação e lê/escreve direto em um arquivo .db no disco. Zero overhead de rede.

2. Os SGBDs em Detalhe

Para cada banco, além do paradigma, explicamos os mecanismos internos que aparecem nas análises dos resultados.

SQLite — Acesso Direto (Embarcado)

PROF

Não existe processo de servidor. O banco é uma **biblioteca** que roda dentro da própria aplicação e lê/escreve diretamente em um arquivo **.db** no disco. Zero overhead de rede ou protocolo.

Conceito-chave: Lock Exclusivo de Escrita

Por padrão, o SQLite usa um mecanismo simples para controlar quem pode escrever: **apenas um escritor por vez**. Quando uma conexão começa a escrever, ela trava o arquivo inteiro — todas as outras que queiram escrever precisam esperar.

```
Conexão 1 escrevendo → arquivo travado
Conexão 2 quer escrever → espera...
Conexão 3 quer escrever → espera...
```

Leituras simultâneas são permitidas sem problema. O colapso aparece quando muitos tentam **escrever ao mesmo tempo** — que é exatamente o Cenário C.

MySQL (InnoDB) — Modelo de Threads

Um processo central recebe todas as conexões. Cada cliente vira uma **thread** dentro desse processo, compartilhando memória — mais leve por conexão que o modelo de processos.

InnoDB — o motor interno do MySQL

O MySQL suporta diferentes "engines" (motores de armazenamento). O **InnoDB** é o padrão e o responsável por tudo que torna o MySQL capaz de lidar com acessos simultâneos: transações, travamentos precisos por linha (não por tabela inteira) e MVCC.

MVCC — múltiplas versões dos dados

Sem MVCC, uma escrita trava o dado para todos os leitores. Com MVCC, ao invés de travar, o banco **cria uma nova versão** do dado modificado e mantém a versão antiga para quem já estava lendo.

```
Sem MVCC:  leitor espera o escritor terminar
Com MVCC:  leitor vê a versão antiga | escritor cria versão nova ←
simultâneos
```

Regra: leitores não bloqueiam escritores. Escritores não bloqueiam leitores.

O MySQL guarda as versões antigas em um espaço separado chamado **undo log**.

fsync — gravação segura no disco

O sistema operacional é "preguiçoso": ele acumula gravações na RAM e só escreve no disco quando achar conveniente. O **fsync** é uma chamada que força a gravação imediata no disco físico — garantindo que o dado sobrevive a uma queda de energia.

O MySQL por padrão chama fsync **a cada commit individual**. Com 100 conexões fazendo commits separados ao mesmo tempo, isso se torna 100 operações de disco simultâneas — o disco vira o gargalo. Esse é o principal motivo pelo qual o MySQL teve p99 de **224 ms** no Cenário C, contra **6 ms** do PostgreSQL.

PostgreSQL — Modelo de Processos

Para cada conexão, cria um **processo separado** no sistema operacional. Maior custo de memória por conexão, mas isolamento total — a falha de um cliente não afeta os demais.

MVCC — mesma ideia, implementação diferente

Assim como o MySQL, o PostgreSQL usa MVCC para permitir leituras e escritas simultâneas. A diferença está em onde as versões antigas ficam guardadas: no PostgreSQL, elas ficam **direto na tabela principal**,

marcadas como "mortas". Não há undo log separado — as versões antigas convivem com as novas no mesmo espaço em disco.

WAL + Group Commit — o segredo do desempenho em escritas concorrentes

O **WAL (Write-Ahead Log)** funciona como um rascunho antes da alteração definitiva: antes de modificar qualquer dado no banco, o PostgreSQL registra a intenção num arquivo de log. Se o servidor cair, ao reiniciar ele lê o log e sabe o que fazer — termina ou desfaz a operação com segurança.

O que torna o PostgreSQL mais rápido que o MySQL no Cenário C é o **group commit**: quando várias transações terminam ao mesmo tempo esperando para confirmar no disco, o PostgreSQL as **agrupa e faz uma única gravação** para todas. O MySQL faz uma gravação separada por transação — muito mais operações de disco no mesmo período.

```
MySQL:      commit 1 → fsync | commit 2 → fsync | commit 3 → fsync ...
PostgreSQL: commit 1 + commit 2 + commit 3 → um único fsync
```

3. Como testamos

Estrutura do benchmark

- **Dataset:** catálogo de ~123 mil álbuns musicais (RYM Top 5000), usado como dados reais para inserção e consulta.
- **Scripts:** um `script.py` por banco, todos com os mesmos parâmetros para garantir comparação justa.
- **Infraestrutura:** PostgreSQL e MySQL rodam em containers Docker; SQLite acessa um arquivo em volume local.

Parâmetros de execução

Parâmetro	Valor
Operações por rodada (INSERTs ou SELECTs)	100.000
Conexões simultâneas (cenários B e C)	100 workers
Operações por worker (B e C)	1.000
Repetições por experimento	10 (1 descartada como aquecimento, 9 consideradas)

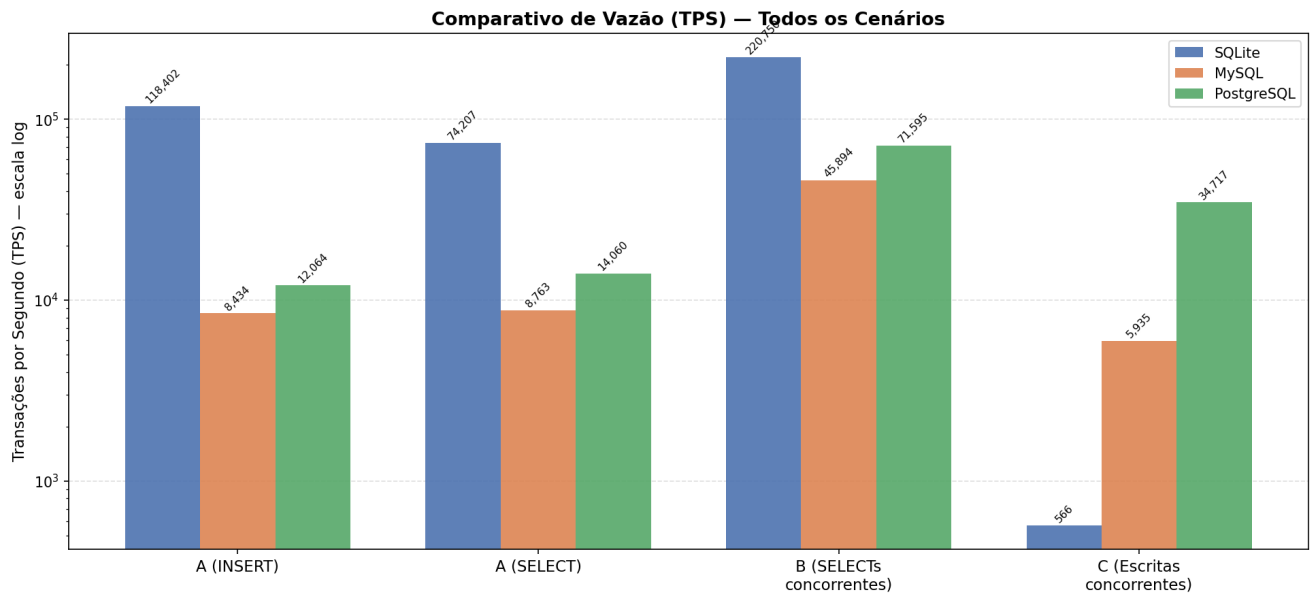
Ambiente

- **Máquina:** Ryzen 5 5600X, 32 GB RAM, Linux Ubuntu 25.10 x86_64
- **Rede:** comunicação via rede interna Docker (localhost) — sem latência de rede real entre os containers

4. Hipóteses esperadas

Cenário	O que esperávamos
A — Carga isolada	SQLite vence com folga: sem servidor, sem rede, acesso direto ao arquivo.
B — Leituras concorrentes	MySQL deveria se sair melhor que PostgreSQL (threads vs. processos). SQLite começaria a criar gargalos.
C — Escritas concorrentes	PostgreSQL lida melhor (MVCC robusto). SQLite entra em colapso com erros de "database is locked".

5. Resultados



Leitura do gráfico acima: escala logarítmica. Cada grupo de barras é um cenário. Quanto mais alta a barra, mais transações por segundo — melhor desempenho.

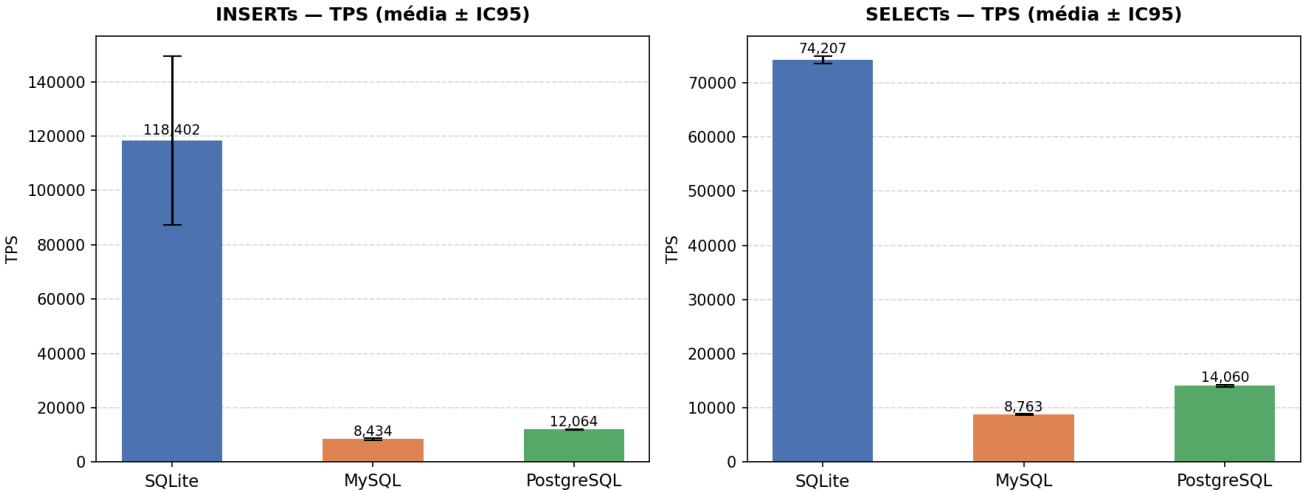
Cenário A — Carga Isolada

PROF

O que é: uma única conexão inserindo 100k registros e depois lendo 100k registros. Sem concorrência.

O que observar: TPS (mais alto = mais rápido) e latência por operação (mais baixo = mais responsivo).

Cenário A — Carga Isolada (1 Conexão, 100k ops)



Resultados:

Banco	INSERT TPS	SELECT TPS	Lat. p99 INSERT	Lat. p99 SELECT
SQLite	118.402	74.207	0,01 ms	0,02 ms
PostgreSQL	12.064	14.060	0,11 ms	0,09 ms
MySQL	8.434	8.763	0,14 ms	0,15 ms

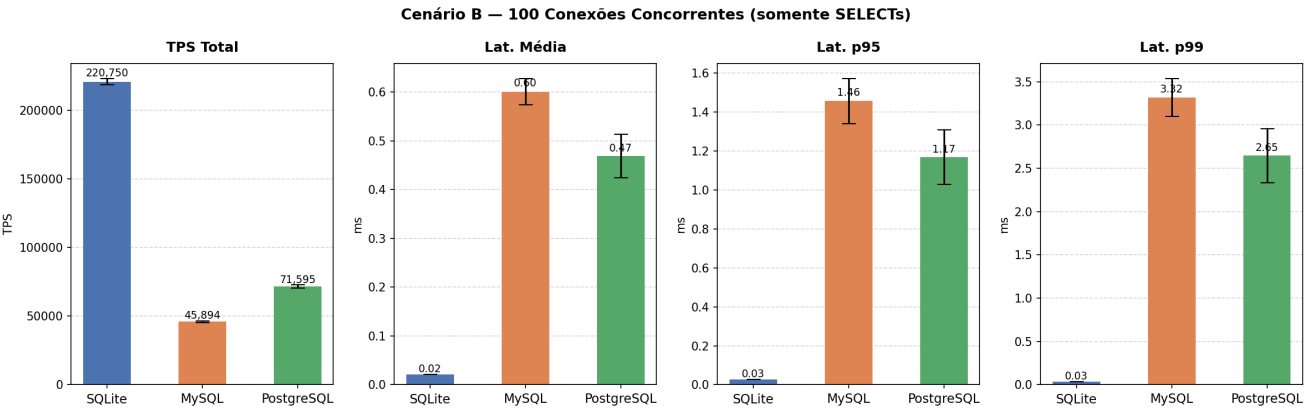
✓ Hipótese confirmada.

SQLite foi ~10x mais rápido que PostgreSQL e ~14x mais rápido que MySQL em INSERTs. A latência sub-milissegundo do SQLite confirma a vantagem do acesso direto ao arquivo: sem socket, sem protocolo de rede, sem processo servidor intermediário.

Cenário B — Leituras Concorrentes

O que é: 100 conexões simultâneas fazendo apenas SELECTs ao mesmo tempo.

O que observar: TPS total do sistema e latência — principalmente o p95 e p99, que mostram o quão "sofrido" foi para os clientes mais lentos.



Resultados:

Banco	TPS	Lat. média	Lat. p95	Lat. p99
SQLite	220.750	0,02 ms	0,03 ms	0,03 ms
PostgreSQL	71.595	0,47 ms	1,17 ms	2,65 ms
MySQL	45.894	0,60 ms	1,46 ms	3,32 ms

△ Hipótese parcialmente errada — resultado mais rico que o esperado.

Dois pontos divergiram:

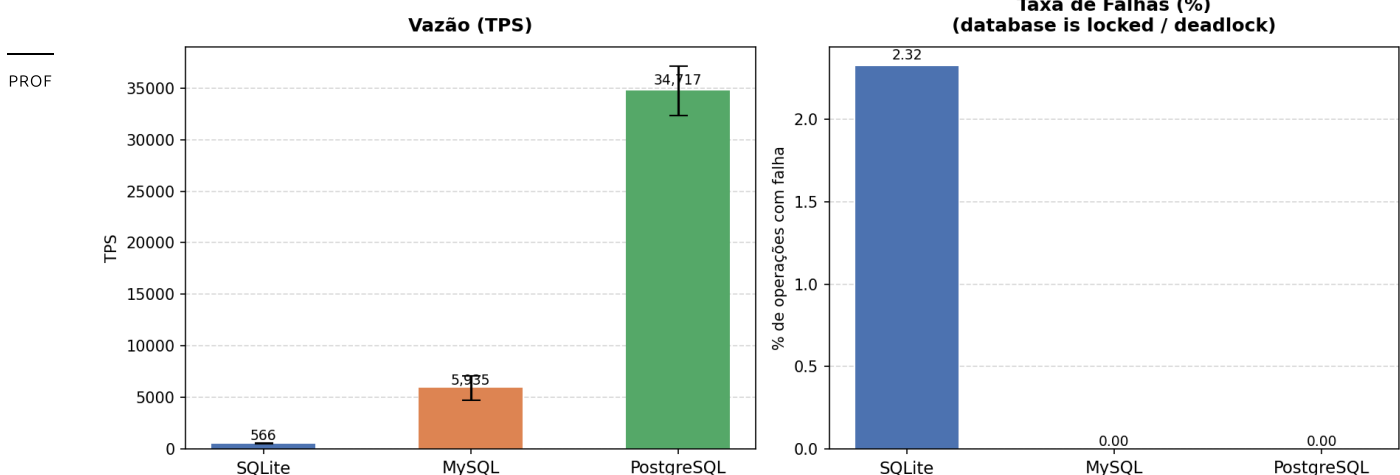
1. **SQLite não criou gargalo — dominou.** O SQLite permite múltiplos leitores simultâneos sem trava. Com 100 processos lendo o mesmo arquivo (que cabe inteiro na RAM), o sistema operacional serviu todos da memória sem overhead de rede. O paradigma embarcado foi ainda mais vantajoso aqui do que esperávamos.
2. **PostgreSQL bateu MySQL, não o contrário.** A hipótese era que threads (MySQL) seriam mais leves que processos (PostgreSQL) para muitas conexões. Os dados refutaram essa previsão — mas não temos uma causa confirmada. O que podemos afirmar é que a vantagem de threads em custo de conexão **não foi o gargalo dominante** a 100 conexões: se fosse, o MySQL teria vencido. A causa exata — seja diferenças de eficiência no protocolo de rede, na criação de snapshots MVCC por leitura, ou nos detalhes internos do gerenciador de buffer — exigiria profiling mais detalhado, que está fora do escopo deste benchmark.

Cenário C — Escritas Concorrentes

O que é: 100 conexões simultâneas fazendo INSERTs e UPDATEs ao mesmo tempo.

O que observar: TPS, taxa de falhas (operações que o banco recusou) e latência de escrita.

Cenário C — 100 Conexões Concorrentes (Escritas: 50% INSERT + 50% UPDATE)



Resultados:

Banco	TPS	Falhas	Lat. p99 INSERT	Lat. p99 UPDATE
-------	-----	--------	-----------------	-----------------

Banco	TPS	Falhas	Lat. p99 INSERT	Lat. p99 UPDATE
PostgreSQL	34.717	0%	6 ms	6 ms
MySQL	5.935	0%	224 ms	343 ms
SQLite	566	2,32%	9 ms	8 ms

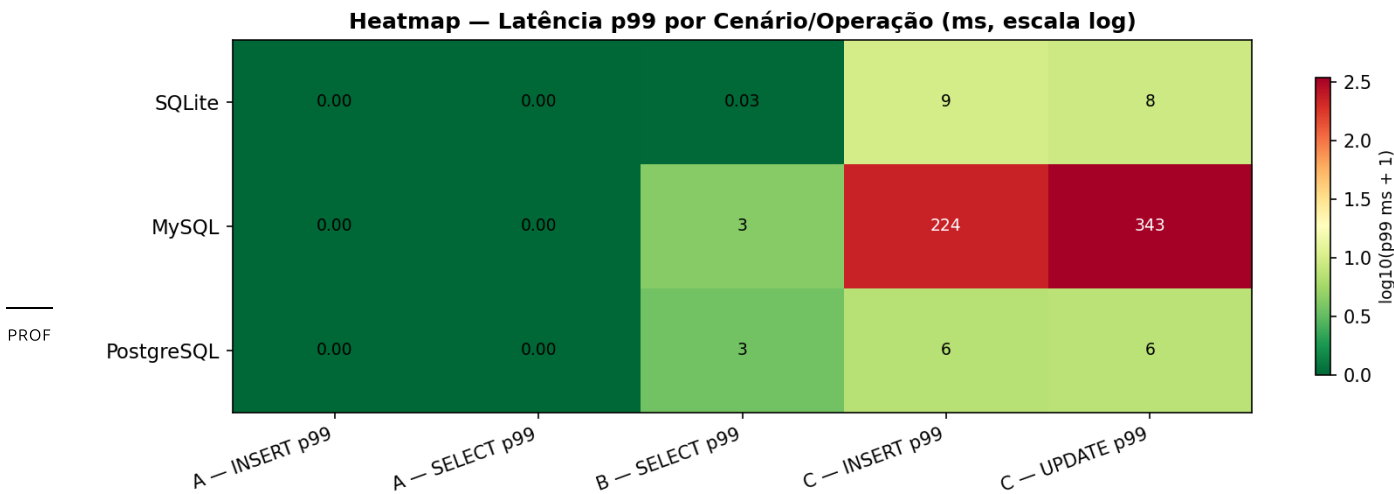
✓ Hipótese confirmada — com surpresas no tamanho da diferença.

- **SQLite travou como esperado.** Por padrão, o SQLite aceita apenas um escritor por vez. Com 100 tentando ao mesmo tempo, 99 ficam na fila. Se não conseguem o acesso em 5 segundos, falham. Resultado: 566 TPS e 2,32% das operações descartadas.
- **PostgreSQL ganhou de MySQL por ~6x**, não por uma margem pequena. Ambos têm MVCC e zeraram as falhas, mas o PostgreSQL foi muito superior em throughput. Um dos motivos: o MySQL por padrão força um **fsync** no disco a cada commit individual — e nesse benchmark cada operação tem seu próprio commit, tornando isso muito custoso.

6. Análises adicionais

Heatmap de pior caso (p99) por banco e cenário

O gráfico abaixo mostra, de uma vez só, onde cada banco teve seus piores momentos. **Verde = latência baixa. Vermelho = latência alta.**



O padrão visual confirma tudo que vimos:

- Nos cenários A e B, os três bancos aparecem em verde — todos têm latências baixas. O SQLite tem os menores valores, mas MySQL e PostgreSQL também estão na faixa de milissegundos.
- No cenário C, o contraste aparece: **PostgreSQL permanece verde** (6 ms), **MySQL fica vermelho** (224–343 ms de p99), reflexo das filas de commit. O colapso do SQLite nesse cenário é mais visível no gráfico de TPS e taxa de falhas — o heatmap mostra apenas a latência das operações que completaram com sucesso (~9 ms), não as que falharam por timeout.

7. Conclusão

Os três paradigmas se comportaram exatamente como a teoria de banco de dados prevê — cada um com seu nicho claro:

Paradigma	Quando vence	Quando perde
SQLite (embarcado)	Usuário único ou leituras concorrentes locais: velocidade incomparável sem nenhum overhead de rede	Qualquer cenário com escritas concorrentes: o modelo de lock exclusivo serializa tudo
MySQL (threads)	Potencialmente vantajoso em cenários com milhares de conexões simultâneas, onde threads consomem menos memória que processos	A 100 conexões não superou o PostgreSQL em nenhum cenário; custo de commit individual é alto
PostgreSQL (processos)	Escritas concorrentes pesadas: MVCC e controle de transações robusto levam a 6x mais TPS que o MySQL no cenário C	Uso de memória por conexão mais alto (um processo por cliente)

A mensagem central: a escolha do SGBD não é sobre "qual é o mais rápido" em termos absolutos — é sobre qual paradigma se encaixa no padrão de acesso da aplicação. SQLite para uso local/embarcado, PostgreSQL para workloads de escrita concorrente.

Ressalva sobre o MySQL: dentro do escopo deste benchmark (100 conexões), o MySQL não superou o PostgreSQL em nenhum dos cenários testados. A principal vantagem teórica do modelo de threads — menor consumo de memória por conexão — pode se tornar relevante em aplicações com milhares de conexões simultâneas, cenário que ficou fora do escopo deste trabalho.

8. Referências

SQLite

[1] D. Richard Hipp et al. **SQLite — File Locking And Concurrency In SQLite Version 3**. SQLite Consortium, 2010–2024. Disponível em: <https://www.sqlite.org/lockingv3.html>

[2] D. Richard Hipp et al. **SQLite — About SQLite**. SQLite Consortium. Disponível em: <https://www.sqlite.org/about.html>

MySQL / InnoDB

[3] Oracle Corporation. **MySQL 8.0 Reference Manual — Introduction to InnoDB**. Oracle, 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/innodb-introduction.html>

[4] Oracle Corporation. **MySQL 8.0 Reference Manual — InnoDB Multi-Versioning**. Oracle, 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/innodb-multi-versioning.html>

[5] Oracle Corporation. **MySQL 8.0 Reference Manual — InnoDB Undo Logs**. Oracle, 2024. Disponible em: <https://dev.mysql.com/doc/refman/8.0/en/innodb-undo-logs.html>

[6] Oracle Corporation. **MySQL 8.0 Reference Manual — innodb_flush_log_at_trx_commit**. Oracle, 2024. Disponible em: https://dev.mysql.com/doc/refman/8.0/en/innodb-parameters.html#sysvar_innodb_flush_log_at_trx_commit

PostgreSQL

[7] The PostgreSQL Global Development Group. **PostgreSQL Documentation — How Connections Are Established**. PostgreSQL, 2024. Disponible em: <https://www.postgresql.org/docs/current/connect-estab.html>

[8] The PostgreSQL Global Development Group. **PostgreSQL Documentation — Introduction to MVCC**. PostgreSQL, 2024. Disponible em: <https://www.postgresql.org/docs/current/mvcc-intro.html>

[9] The PostgreSQL Global Development Group. **PostgreSQL Documentation — Reliability and the Write-Ahead Log**. PostgreSQL, 2024. Disponible em: <https://www.postgresql.org/docs/current/wal.html>

[10] The PostgreSQL Global Development Group. **PostgreSQL Documentation — WAL Configuration (Asynchronous Commit / Group Commit)**. PostgreSQL, 2024. Disponible em: <https://www.postgresql.org/docs/current/wal-async-commit.html>